A Graph-Based Mixed-Integer Formulation for Spiral Galaxies and Center-Guided Region Partitioning

Santiago Florido Gomez
RO203, Diplôme d'ingénieur
ENSTA Paris
Palaiseau, France
santiago.florido@ensta.fr

Jose Daniel Chacon Gomez
RO203, Diplôme d'ingénieur
ENSTA Paris
Palaiseau, France
jose-daniel.chacon@ensta.fr

***Abstract*—This paper presents the development of a graph-based exact formulation for the *Spiral Galaxies* puzzle and studies its relevance as a model for center-guided connected region partitioning. The board is represented as a grid graph in which each feasible galaxy is defined as a connected region assigned to a unique center and constrained by admissibility and central symmetry. The resulting problem is formulated as a mixed-integer linear model combining binary assignment variables with flow-based constraints to enforce connectivity. Beyond the mathematical model, the project includes a Python implementation of two complementary solver backends, validation tools for feasibility checks, a fixed-range dataset-generation pipeline, an empirical hard-threshold search utility driven by the exact solver, and a Pygame interface for interactive visualization and experimentation. The article describes the modeling choices, the software architecture used to translate the formulation into code, and the role of this framework as a compact case study in exact discrete optimization with geometric priors.**

***Index Terms*—graph theory, constraint solving, puzzle modeling, pygame**

I. Introduction

Partitioning a discrete domain into regions subject to a prescribed topological geometry, with unique centers that must be contained within each region, requires that every subregion be connected. In addition, imposing a central symmetry constraint constitutes the basic problem structure underlying the *Spiral Galaxies* puzzle [1]. However, this formulation is not limited to the puzzle itself; rather, it represents a broader scientific problem with considerable academic interest.

From a modeling perspective, this places the project at the intersection of graph theory, computational geometry, and mixed-integer optimization. Once the board is represented as a grid graph, each feasible galaxy can be interpreted as a connected subset of vertices anchored at a single center and closed under a symmetry operator. Problems of this kind extend well beyond recreational puzzles. Graph partitioning with connectivity constraints appears, for instance, in image segmentation, where pixels or superpixels are grouped into coherent regions on an adjacency graph [2]. Related contiguity-constrained partitioning also appears in political districting [3]. More generally, connected graph partitioning is a well-studied NP-hard family of problems with applications in image processing, cluster analysis, robotics, and operating systems [4].

More specifically, the complete structure of the *Spiral Galaxies* problem is particularly relevant to the separation of touching cell nuclei in histopathological or microscopy images [7]. In this setting, one starts from a binary foreground mask containing several adjacent or overlapping nuclei, together with an estimate of their centers. The objective is to divide this mask into individual instances, assigning one connected region to each detected center. This shares the same structural principle as the galaxy problem, except that, in this biomedical context, exact central symmetry is not an intrinsic requirement of each instance, even though symmetry cues and shape priors can still improve segmentation quality [5], [6].

Under this interpretation, each cell can be modeled as a galaxy, the image grid corresponds to the puzzle board, and the detected nuclear centroids play the role of the puzzle centers. Solving the partitioning problem is therefore equivalent to reconstructing a valid decomposition of the foreground into connected and geometrically regular regions, each associated with a unique center.

Consequently, the project is useful not only as a playable application, but also as a compact case study for evaluating how geometric priors can be encoded as linear constraints and how exact optimization solvers can recover globally valid region decompositions.

II. Mathematical Model

In order to define the problem to be modeled in a clear and mathematically precise way, it is first necessary to establish the rules of the game and describe the dynamics of each instance. This provides the basis for subsequently formulating the constraints that enforce those rules and give rise to the optimization system solved for each instance.

A. Combinatorial Rules of a Valid Spiral Galaxies Instance

The *Spiral Galaxies* puzzle is defined on a rectangular grid of unit cells together with a set of prescribed galaxy centers [1]. A valid solution must satisfy the following rules:

- Each cell must be assigned to exactly one galaxy. The set of galaxies therefore forms an exact partition of the board.
- Each galaxy is associated with one prescribed center. A center may lie in the interior of a cell, on the midpoint of an edge, or on a grid vertex.
- Each galaxy must contain its own center. If the center lies on a cell, an edge, or a vertex, then the corresponding incident one, two, or four cells must belong to that galaxy.
- Each galaxy must be connected under orthogonal adjacency. Any two cells of the same galaxy must be linked by a path of side-adjacent cells belonging to that same region.
- Each galaxy must be invariant under a rotation of $180°$ around its center. Thus, whenever a cell belongs to a galaxy, its rotated counterpart must also belong to that galaxy whenever the rotated position remains inside the board.

Taken together, these conditions define the game as a partitioning problem on a discrete domain with center incidence, connectivity, and central symmetry constraints. This rule set provides the basis for the mathematical formulation introduced next.

B. Combinatorial and Geometric Modeling

The universe of cells composing the board is defined in order to obtain a discrete representation of the playing domain. Let

$$\mathcal{U} := [N] \times [M]. \tag{1}$$

In Eq. 1, N denotes the number of rows, M denotes the number of columns, and $[N] \times [M]$ represents the set of all ordered pairs (i, j) such that $1 \leq i \leq N$ and $1 \leq j \leq M$.

Orthogonal adjacency is modeled through the grid graph

$$\mathcal{H} = (\mathcal{U}, \mathcal{E}), \qquad \mathcal{E} := \{\{u, v\} \subseteq \mathcal{U} : \|u - v\|_1 = 1\}. \tag{2}$$

In Eq. 2, $\mathcal{H}$ denotes the grid graph, $\mathcal{U}$ is the set of nodes, and $\mathcal{E}$ is the set of edges. The term $\|u - v\|_1$ represents the Manhattan distance between cells u and v. Therefore, the condition $\|u - v\|_1 = 1$ indicates that the two cells share one side. The connectivity of each galaxy is defined precisely over the graph described in Eq. 2.

The centers associated with the galaxies encode the geometric prior of the system and are modeled as follows:

$$\begin{aligned} \mathcal{G} &:= \{1, 2, \ldots, k\}, \\ L(g) &= (a_g, b_g), \\ a_g &\in \left\{1, \tfrac{3}{2}, 2, \tfrac{5}{2}, \ldots, N\right\}, \\ b_g &\in \left\{1, \tfrac{3}{2}, 2, \tfrac{5}{2}, \ldots, M\right\}. \end{aligned} \tag{3}$$

In Eq. 3, k denotes the total number of centers, g indexes a specific center, and $L(g)$ represents the geometric position of center g. The quantities a_g and b_g denote its vertical and horizontal coordinates, respectively. Integer coordinates indicate that the center lies inside a cell, whereas half-integer coordinates indicate that the center lies on an edge or on a vertex of the grid.

The need therefore arises to define the rotational geometry within each region into which the domain is partitioned. This makes it possible to express the symmetry constraint of the game in a natural and mathematically precise way. For this purpose, the $180°$ rotation around center g induces the operator

$$\tau_g(i, j) := (2a_g - i,\ 2b_g - j). \tag{4}$$

In Eq. 4, (i, j) denotes the original cell, while $\tau_g(i, j)$ denotes its symmetric counterpart with respect to center g. The term $2a_g - i$ reflects the row coordinate with respect to the center, and $2b_g - j$ reflects the column coordinate with respect to the center. The symmetry of a galaxy is therefore enforced by requiring that, if a cell belongs to the galaxy associated with center g, then its image under the operator τ_g must also belong to that same galaxy.

Not all cells can belong to a given galaxy, since their symmetric counterpart may lie outside the board. In this context, it is necessary to define the admissible domain associated with each center. Thus,

$$\mathcal{U}_g := \{u \in \mathcal{U} : \tau_g(u) \in \mathcal{U}\}. \tag{5}$$

In Eq. 5, $\mathcal{U}_g$ denotes the set of cells that are admissible for center g. Therefore, $u \in \mathcal{U}_g$ means that both cell u and its symmetric counterpart $\tau_g(u)$ lie within the board.

Considering the kernel as the set of mandatory cells geometrically associated with center g, the kernel of center g is defined as

$$K(g) := \{(i, j) \in \mathcal{U} : i \in \{\lfloor a_g \rfloor, \lceil a_g \rceil\},\ j \in \{\lfloor b_g \rfloor, \lceil b_g \rceil\}\}. \tag{6}$$

In Eq. 6, $K(g)$ denotes the kernel associated with center g. The operator $\lfloor a_g \rfloor$ represents the lower integer part of a_g, while $\lceil a_g \rceil$ represents its upper integer part. The same interpretation applies analogously to b_g.

The cardinality of $K(g)$ depends on the geometric position of the center. If $a_g, b_g \in \mathbb{Z}$, then $|K(g)| = 1$. If exactly one coordinate is half-integer, then $|K(g)| = 2$. If both coordinates are half-integers, then $|K(g)| = 4$. If X_g is a connected and $180°$-symmetric region that geometrically contains the center $L(g)$, then

$$K(g) \subseteq X_g. \tag{7}$$

In Eq. 7, X_g denotes the galaxy associated with center g, and $K(g) \subseteq X_g$ means that all cells in the kernel are mandatory.

This allows a combinatorial definition of a valid solution for a given instance of the game [1]. A solution of the puzzle is a family of sets $\{X_g\}_{g\in\mathcal{G}}$ satisfying

$$\bigcup_{g\in\mathcal{G}} X_g = \mathcal{U}, \tag{8}$$

$$X_g \cap X_h = \emptyset \qquad \forall g,h \in \mathcal{G},\ g \neq h, \tag{9}$$

$$K(g) \subseteq X_g \qquad \forall g \in \mathcal{G}, \tag{10}$$

$$u \in X_g \Rightarrow \tau_g(u) \in X_g \qquad \forall g \in \mathcal{G},\ \forall u \in X_g, \tag{11}$$

$$\mathcal{H}[X_g] \text{ is connected} \qquad \forall g \in \mathcal{G}. \tag{12}$$

Eq. 8 states that all cells of the board must be covered. Eq. 9 ensures that no cell belongs to two distinct galaxies. Eq. 10 enforces the geometric containment of the corresponding center. Eq. 11 imposes the rotational symmetry condition. Finally, $\mathcal{H}[X_g]$ denotes the subgraph of $\mathcal{H}$ induced by X_g, and Eq. 12 enforces side-connectivity for each galaxy.

C. MILP with Binary Variables Formulation

In order to implement the combinatorial model described above, it is necessary to introduce binary variables that represent both the assignment structure of a game instance and the state of its constraints. Thus, the binary variables are defined as

$$x_{u,g} \in \{0,1\} \qquad \forall u \in \mathcal{U},\ \forall g \in \mathcal{G}. \tag{13}$$

The assignment semantics of these variables is given by

$$x_{u,g} = \begin{cases} 1, & \text{if } u \text{ is assigned to center } g, \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

In Eq. 14, $x_{u,g} = 1$ denotes an active assignment, whereas $x_{u,g} = 0$ indicates that cell u is not assigned to center g.

Each cell of the board must be assigned exclusively to exactly one center. This condition is imposed by the constraint

$$\sum_{g\in\mathcal{G}} x_{u,g} = 1 \qquad \forall u \in \mathcal{U}. \tag{15}$$

In Eq. 15, the term $\sum_{g\in\mathcal{G}} x_{u,g}$ aggregates all possible assignments of cell u over the set of centers. The value 1 on the right-hand side enforces a unique assignment for each cell.

To perform this assignment consistently, it is necessary to define the validity of the image of a cell with respect to a particular center. If a cell does not have a valid symmetric counterpart with respect to center g, then it cannot belong to the corresponding galaxy. This condition is expressed as

$$x_{u,g} = 0 \qquad \forall g \in \mathcal{G},\ \forall u \in \mathcal{U} \setminus \mathcal{U}_g. \tag{16}$$

In Eq. 16, $\mathcal{U} \setminus \mathcal{U}_g$ denotes the set of cells whose symmetric counterpart lies outside the board with respect to center g. Therefore, the constraint eliminates assignments that are geometrically infeasible.

The rotational symmetry condition is imposed through the constraint

$$x_{u,g} = x_{\tau_g(u),g} \qquad \forall g \in \mathcal{G},\ \forall u \in \mathcal{U}_g. \tag{17}$$

In Eq. 17, $x_{u,g}$ represents the membership of cell u in the galaxy associated with center g, while $x_{\tau_g(u),g}$ represents the membership of its symmetric counterpart. The equality enforces that both cells are either included in or excluded from the same galaxy simultaneously.

Since the cells in the kernel must necessarily belong to the galaxy associated with their corresponding center, this requirement must be explicitly enforced within the model. This is expressed as

$$x_{u,g} = 1 \qquad \forall g \in \mathcal{G},\ \forall u \in K(g). \tag{18}$$

In Eq. 18, $K(g)$ denotes the set of cells incident to center g. Setting the corresponding variables equal to 1 fixes all kernel cells inside the galaxy associated with center g.

To enforce connectivity, a flow-based formulation is introduced for each galaxy [3], [4]. Let $\overrightarrow{\mathcal{E}}$ denote the directed version of the edge set $\mathcal{E}$, obtained by replacing each undirected edge $\{u,v\} \in \mathcal{E}$ with the two directed arcs (u,v) and (v,u). For each center g, an artificial source node s_g is connected to the cells in the kernel $K(g)$. The corresponding arc set is defined as

$$\mathcal{A}_g := \overrightarrow{\mathcal{E}} \cup \{(s_g, u) : u \in K(g)\}. \tag{19}$$

A nonnegative flow variable is then associated with each arc used to connect the galaxy:

$$F_a^g \geq 0 \qquad \forall g \in \mathcal{G},\ \forall a \in \mathcal{A}_g. \tag{20}$$

In Eq. 20, F_a^g denotes the amount of flow sent through arc a in order to connect the galaxy associated with center g.

Let

$$n := |\mathcal{U}| = NM. \tag{21}$$

In Eq. 21, n denotes the total number of cells in the board and provides a valid upper bound for the flow variables. For each directed arc $(u,v) \in \overrightarrow{\mathcal{E}}$, the following capacity constraints are imposed:

$$F_{u,v}^g \leq n x_{u,g} \qquad \forall g \in \mathcal{G},\ \forall (u,v) \in \overrightarrow{\mathcal{E}}, \tag{22}$$

$$F_{u,v}^g \leq n x_{v,g} \qquad \forall g \in \mathcal{G},\ \forall (u,v) \in \overrightarrow{\mathcal{E}}. \tag{23}$$

In Eqs. 22 and 23, $F_{u,v}^g$ represents the flow sent from cell u to cell v. These constraints activate an arc only when both endpoint cells belong to the galaxy associated with center g.

The capacity of the arcs leaving the artificial source is defined as

$$F^g_{s_g,u} \leq n x_{u,g} \qquad \forall g \in \mathcal{G},\ \forall u \in K(g). \tag{24}$$

In Eq. 24, $F^g_{s_g,u}$ denotes the flow entering a kernel cell from the artificial source. If $x_{u,g} = 0$, no flow can be sent through the corresponding source arc. Let $\delta^+(u)$ and $\delta^-(u)$ denote the sets of outgoing and incoming arcs of cell u in $\vec{\mathcal{E}}$, respectively.

Each selected cell must consume one net unit of flow. This condition is imposed by the flow-balance constraint

$$\begin{aligned}\sum_{a\in\delta^-(u)} F^g_a + \mathbf{1}_{\{u\in K(g)\}} F^g_{s_g,u} - \sum_{a\in\delta^+(u)} F^g_a \\ = x_{u,g}, \qquad \forall g \in \mathcal{G},\ \forall u \in \mathcal{U}.\end{aligned} \tag{25}$$

In Eq. 25, the first summation represents the flow entering cell u from adjacent cells, while the second summation represents the flow leaving u toward adjacent cells. The term $\mathbf{1}_{\{u\in K(g)\}}$ is an indicator function equal to 1 when $u \in K(g)$ and equal to 0 otherwise. The right-hand side $x_{u,g}$ forces each cell assigned to galaxy g to absorb exactly one unit of flow.

Finally, the total amount of flow emitted by the artificial source must be equal to the number of cells assigned to the corresponding galaxy:

$$\sum_{u\in K(g)} F^g_{s_g,u} = \sum_{u\in\mathcal{U}} x_{u,g} \qquad \forall g \in \mathcal{G}. \tag{26}$$

In Eq. 26, the left-hand side represents the total flow emitted from the artificial source associated with center g, whereas the right-hand side gives the total number of cells assigned to that galaxy. This equality ensures that every selected cell is reachable from the kernel through active arcs, thereby enforcing connectivity.

D. Feasibility Interpretation and Complete MILP

The *Spiral Galaxies* puzzle is not, in its original form, a genuine optimization problem. Rather, it is a combinatorial feasibility problem: the goal is to determine whether there exists an assignment of cells to centers that satisfies coverage, admissibility, symmetry, kernel inclusion, and connectivity simultaneously. For compatibility with standard MILP solvers, the problem is therefore written in equivalent optimization form with a null objective function.

Using the notation introduced above, the complete MILP can then be written as

$$\min \quad 0 \tag{27a}$$

$$\begin{aligned}\text{s.t.} \quad & (15),\ (16),\\ & (17),\ (18),\\ & (22),\ (23),\\ & (24),\ (25),\ (26),\\ & (13),\ (20).\end{aligned} \tag{27b}$$

Thus, any feasible solution satisfying Eq. (27a) together with the constraint set referenced in Eq. (27b) defines a valid solution of the puzzle. Conversely, if the model is infeasible, then the corresponding game instance admits no valid decomposition under the imposed rules.

III. Methodology

It is therefore necessary to translate the mathematical model described above into a practical implementation capable of encoding the geometric and combinatorial constraints of the puzzle. This implementation connects the formal formulation with a discrete software representation of the board, graph-based data structures, structural validation procedures, exact MILP formulations, and a constructive generation pipeline.

The puzzle is represented as an exact discrete board with precise coordinates. From this representation, the symmetry-aware geometric operators, admissible domains, kernel cells, and orthogonal adjacency graph are derived. These objects are then compiled into a cached instance representation that serves as the shared semantic basis for validators, MILP builders, solver services, generation routines, certification logic, difficulty calibration, and persistence tooling.

A. Discrete Domain and Geometric Invariants

The board is represented as a finite set of cells implemented through the `Cell` structure, which corresponds to the computational realization of the discrete atom denoted in the mathematical model as a board cell. This object is immutable and orderable, ensuring that cells can be handled consistently across the different components of the implementation.

The second element of this discretization is `BoardSpec`. This structure does not merely define the dimensions N and M; rather, it specifies the finite support on which the set of cells is defined. In addition, for the validation of the model constraints, it is important to fix a deterministic traversal of this cell set. For this reason, through `iter_cells()` and `cells()`, `BoardSpec` defines a canonical enumeration of all cells in the board.

Each center has a geometric position, which is the information used, as described in the mathematical constraints, to determine symmetries, admissible domains, and galaxy kernels. This information is stored in `CenterSpec`. It is also important to note that the coordinates are not stored as floating-point values. Instead, they are represented as integer coordinates on a doubled lattice. This representation is fundamental because a center may lie inside a cell, on an edge, or on a vertex, and this affects the way numerical comparisons interact with kernel construction and admissible cells under rotation.

The doubled lattice converts integer and half-integer coordinates into a single exact integer system. This eliminates rounding errors and ensures that all subsequent geometric operations are fully deterministic.

B. Geometric Operators and Symmetry Primitives

The function `tau()` is implemented to represent the discrete rotation operator around a center in the exact doubled-coordinate system. The function `twin_cell()` lifts this purely geometric transformation into a board-aware operation by returning the rotated cell only when its image remains inside the board, and returning `None` otherwise.

The functions `is_admissible_cell()` and `admissible_cells()` construct the complete set of cells that can belong to the galaxy associated with a given center. Similarly, `is_kernel_cell()` and `kernel_cells()` encode the mandatory incidence structure around the center, thereby translating the geometric containment condition of the mathematical model into deterministic software procedures.

C. Graph Representation and Cached Puzzle State

The board abstraction is implemented as a graph in order to provide a software realization of the orthogonal adjacency structure defined in the mathematical model. This is achieved through the `GridGraph` class. The method `build_neighbors()` computes the neighbor map for every cell only once, while `_build_edges()` derives the corresponding undirected edge set. In addition, methods such as `induced_neighbors()`, `traverse_component()`, `connected_components()`, and `is_connected()` provide the rest of the codebase with a uniform and executable notion of connectedness.

The structure `PuzzleData` is then defined as the internal compiled representation of the entire puzzle instance. It computes the canonical tuple of cells, records the ordered tuple of centers, builds the `GridGraph`, stores a read-only view of the neighborhood relation, and precomputes, for every center, its admissible domain, its kernel cells, and its twin map. Consequently, `PuzzleData` acts as the shared semantic layer from which validation routines, optimization models, solver interfaces, and generation procedures operate.

D. Structural Validation Layer

An executable predicate logic is implemented to validate the satisfaction of the game rules independently from the optimization solver. This validation layer is mainly used by the generator, which requires a mechanism to reject invalid structures before attempting to compute a solution with the solver. It also serves as a second validation stage for solver outputs, ensuring that the returned assignments satisfy the structural requirements of the puzzle. The function `_prepare_assignment()` defines the normalization boundary of the validator logic. It receives external assignment representations, converts them into a canonical internal form, verifies that all cells and center identifiers are known, and ensures that downstream predicates operate only on well-formed data.

The predicate `partition_is_valid()` checks the exact-cover condition and forbids overlaps between galaxies. In addition, the predicates `admissibility_is_valid()`, `symmetry_is_valid()`, `kernel_is_valid()`, and `connectivity_is_valid()` validate, respectively, admissibility, rotational symmetry, kernel containment, and side-connectivity. Finally, `validate_assignment()` aggregates all these checks into an `AssignmentValidationResult`. This design preserves diagnostic information for each rule instead of reducing the validation process to a single Boolean value, thereby making invalid assignments easier to analyze, debug, and reject during generation or certification.

E. MILP

The main purpose is to represent the local and assignment-level rules of the puzzle as sparse linear algebra over binary variables. For this reason, the constraint set is separated into two components: the constraints that can be expressed directly through local linear relations, and the connectivity constraints, which require a more elaborate global formulation.

1) Base: The central abstraction in this phase is `BaseMilpModel`, since it encapsulates the translation of the base mathematical model into a form that can be solved by `SciPy`. The method `BaseMilpModel.from_puzzle_data()` begins by fixing a canonical ordering for the binary variables $x_{u,g}$, creating one variable for each pair $(\texttt{cell}, \texttt{center.id})$. This design decision directly determines the column ordering of the solver decision vector. From this point on, each family of constraints is not constructed immediately as a complete matrix, but rather as a collection of sparse rows. Each row is represented by a triplet $(\texttt{columns}, \texttt{coefficients}, \texttt{target_value})$, where `columns` specifies the positions of the variables with nonzero coefficients, `coefficients` stores the corresponding coefficient values, and `target_value` represents the right-hand side of the equation.

For example, the partition rows implement Eq. (15) by placing coefficient 1 in every column associated with the same cell u across all centers and by setting the target value equal to 1. Similarly, the symmetry rows implement Eq. (17) by using exactly two columns, corresponding to $x_{u,g}$ and $x_{\tau_g(u),g}$, with coefficients 1 and -1, respectively. In the same way, the inadmissibility and kernel rows are sparse one-variable equalities implementing Eqs. (16) and (18). Consequently, the mathematical constraints introduced in Eqs. (15)–(18) are not redefined in software; instead, they are rewritten row by row in a matrix-oriented format.

The method `_build_exact_constraint()` takes this row-wise representation and converts it into a global sparse matrix A, which is first assembled in COO format and then converted to CSC format. In parallel, the method constructs the right-hand-side vector b. Finally, it creates the object `LinearConstraint(matrix, rhs_vector, rhs_vector)`. In the `SciPy` interface, this imposes

$$b \leq Ax \leq b, \tag{28}$$

which is equivalent to the exact equality constraint

$$Ax = b. \tag{29}$$

In this way, the mathematical notation of the model is systematically translated into a sparse matrix structure that can be directly consumed by the MILP solver.

2) Flow: In the flow-based model, the central idea is to extend the base formulation so that it not only decides which cell belongs to which center, but also certifies that each resulting galaxy is connected. While the base model only uses binary variables $x_{u,g}$, where each variable indicates whether cell u is assigned to center g, the flow model introduces additional variables representing circulation over the board adjacency graph.

From an implementation perspective, `FlowMilpModel` takes an already constructed `BaseMilpModel` as its starting point and extends its decision vector with two new families of variables. The first family corresponds to directed flow variables $f_{g,u,v}$, which represent the amount of flow sent from cell u to a neighboring cell v inside the galaxy associated with center g. The second family corresponds to source-flow variables $s_{g,k}$, which represent the flow injected from an artificial source into the kernel cells k of each galaxy. Therefore, the solver decision vector no longer contains only cell-center assignment variables, but also internal flow variables and source-injection variables.

The translation to `SciPy` follows the same sparse matrix logic used in the base model. The existing constraints over the variables $x_{u,g}$ are not rebuilt. Instead, they are expanded with zero-valued columns by means of `_expand_constraint()`, so that they remain valid within the extended decision vector. Additional sparse rows are then introduced to encode the flow constraints.

The arc-capacity rows implement Eqs. (22) and (23). In implementation terms, each row couples one directed flow variable with one assignment variable so that flow can circulate only between cells that are active in the same galaxy. Analogously, the source-capacity rows implement Eq. (24), forcing source injection to occur only through kernel cells.

The key structural block is the cell-wise balance encoded from Eq. (25). For each center and each cell, the implementation builds one sparse row containing all incoming directed-flow variables with coefficient $+1$, the source-flow variable with coefficient $+1$ when the cell belongs to the kernel, all outgoing directed-flow variables with coefficient -1, and finally the assignment variable $x_{u,g}$ with coefficient -1. The target value of that row is 0, which reproduces the equality structure of Eq. (25). In the same way, one additional row per center implements the total source-supply condition given in Eq. (26) by balancing the total injected source flow against the total number of cells assigned to that galaxy.

At the interface level, the equalities inherited from the base model and the flow-balance equalities are converted into `LinearConstraint` objects by `_build_exact_constraint()`, while the capacity inequalities are assembled by `_build_upper_bound_constraint()`, which imposes a constraint of the form $Az \leq u$ through lower bounds equal to $-\infty$ and finite upper bounds. Taken together, the rows implementing Eqs. (22), (23), (24), (25), and (26) transform connectivity into an algebraic property of the optimization model: a disconnected region cannot receive flow consistently from its kernel and is therefore automatically excluded from the feasible solution set.

F. Callback-Style Connectivity Rejection

The implementation also provides a second solving backend, denoted `parallel_callback`, whose purpose is not to replace the canonical flow formulation, but to provide a comparative solution strategy based on the base MILP alone. In this backend, the assignment constraints of Eqs. (15)–(18) are solved first without introducing the flow variables of Eqs. (22)–(26). The resulting incumbent is then decoded into a `GalaxyAssignment` and validated at the graph level through `PuzzleData.graph.is_connected()`.

Operationally, the backend implements a callback-style separation loop. If the decoded incumbent is connected, it is accepted immediately. Otherwise, the backend constructs one exclusion cut over the binary assignment vector and resolves the base MILP. This cut forbids the exact disconnected incumbent while preserving the rest of the feasible region of the base model. The same process is repeated until either a connected assignment is found or the remaining base model becomes infeasible. Therefore, connectivity is enforced here by alternating optimization and graph validation rather than by extending the linear model with flow conservation.

This architecture preserves a clean separation of responsibilities. The base MILP builder remains responsible only for the local algebraic constraints, whereas the callback-style backend is responsible for incumbent inspection, graph-connectedness verification, and no-good cut generation. In the current system, `exact_flow` remains the default exact backend used by the interactive game, by the certification stage of generation, by the empirical hard-threshold search, and by the default dataset-construction workflow, while `parallel_callback` is exposed as an alternative backend for comparative experiments and dataset benchmarking.

G. Constructive Generation

The approach then moves from estimating a solution for a particular puzzle instance to generating the instance itself in a structured and policy-aware manner. This process relies on exact certification to confirm that the final instance is mathematically sound.

A generation policy is defined through `PuzzleGenerationRequest`, which specifies the type of puzzle instance to be generated. This request includes the main generation parameters: the desired difficulty level, the board size, the random seed, and the maximum number of generation attempts. The construction begins with the placement of centers through a subdivision of the board

into rectangular regions. The central abstraction in this stage is `RectangleRegion`, which stores a single rectangle and can derive its area, dimensions, cell set, parity pattern, center type, and corresponding `CenterSpec`. The process starts from a single rectangle corresponding to the entire board. Then, `place_candidate_centers()` recursively refines this initial region until a target number of regions is reached. This target number is not fixed; it is determined by `sample_target_center_count()` according to the selected `DifficultyProfile`.

Once several candidate configurations have been generated, the algorithm does not select one uniformly at random. Instead, each candidate is first evaluated through `_candidate_score()`. This score considers several criteria, including how well the induced partition matches the target mixture of center types, how balanced or regular the regions are, how reasonable the spatial separation between regions is, and how suitable the resulting scaffold is for the desired difficulty profile.

The function `_choose_split_candidate()` then applies a stochastic selection strategy over the best-ranked candidates. In other words, the procedure first filters and scores the candidate configurations, and then selects among the strongest alternatives using controlled randomness. The process starts from an already valid constructive assignment, in which each region still essentially coincides with its original rectangle, and seeks to transform some of these regions into non-rectangular shapes that are more interesting from the perspective of difficulty. To achieve this, the procedure does not modify isolated cells. Instead, it operates on symmetry orbits, that is, pairs of cells that must be moved together in order to preserve the $180°$ rotational symmetry around the corresponding center. Based on this principle, the module iterates over admissible cells of a candidate region, constructs their associated orbits, discards those that already belong entirely to the region, removes those that would affect protected kernel cells, and excludes those that do not maintain local contact with the region. It also verifies that the enlarged region remains connected.

Among the orbits that survive this filtering stage, the algorithm assigns a score combining several criteria: how much the orbit moves the region toward the target irregularity defined by the difficulty profile, how much it expands the bounding box of the region, how far it lies from the geometric center of the region, how many new cells it moves outside the base rectangle, and how strongly it reinforces local adjacency. The algorithm then selects stochastically among the best-ranked options and applies the ownership transfer, updating both the owner of each affected cell and the cell set of each region. This process is repeated only over a subset of regions selected according to the difficulty profile, and with an intensity that is also controlled by that profile. Consequently, at low difficulty levels, this deformation step may be absent or highly limited, whereas at higher difficulty levels, the generator attempts to deform more regions and reach greater levels of irregularity. The final result is not imposed directly as the definitive puzzle solution. Instead, it is converted into a guide for the exact solver: on the one hand, the new preferred assignments are recorded; on the other hand, rectangular assignments that should be avoided are identified. The objective is for the final certified solution to preserve the mathematical validity of the model while adopting a less trivial morphology that is better aligned with the desired difficulty level.

1) Validation and Calibration: The generated result is not treated as a final solution. Instead, the existence of a valid solution for the generated instance is verified in the same way as for any puzzle instance solved by the game engine. After this certification step, a difficulty-validation stage is applied in order to confirm that the instance satisfies the requirements associated with the prescribed difficulty level. The system implements an abstract difficulty label that is associated with several generation and validation criteria. These criteria include how many centers the puzzle should contain, what proportion of centers should lie inside cells, on edges, or on vertices, how much overlap is allowed between admissible domains, how much geometric irregularity is desired, how many non-rectangular regions must appear at minimum, and whether uniqueness or greater structural difficulty should be enforced. The calibration logic extracts a multi-factor description of the instance. It measures the board size, the number of centers, the center-type composition, the overlap between admissible domains, the geometric irregularity of the certified regions, and the computational effort required by the solver. The function `region_irregularity()` is central in this process because it measures how far a region is from saturating its bounding box, making it a practical proxy for distinguishing regular rectangular galaxies from more intricate ones.

The function `_solver_effort_score()` incorporates backend metadata, such as node count and whether guided solving was used. Consequently, computational effort also becomes part of the operational notion of difficulty. The resulting score is compared against the expectations of the active difficulty profile and is then mapped back to a difficulty label, as shown in Fig. 1. This turns calibration into a feedback loop between the intended generation policy and the observed certified behavior. Without this step, difficulty labels would only describe what the generator attempted to create, rather than what the final certified instance actually exhibits.

H. Dataset and persistence layer

An instance representation and architecture are provided to support subsequent batch-processing workflows. The central serialized object is `StoredPuzzleInstance`, which stores not only the board and its centers, but also the metadata associated with the generation process and the difficulty-calibration measurements. Therefore, it contains sufficient information to explain both the structure of the puzzle and the way in which it was classified. The function `generate_instance()` wraps repeated seeded attempts into a single callable generation routine and can be coupled with a profile override so that non-canonical board sizes remain governed by the same difficulty policy as the requested family while allowing the center-count range to scale with the board area.

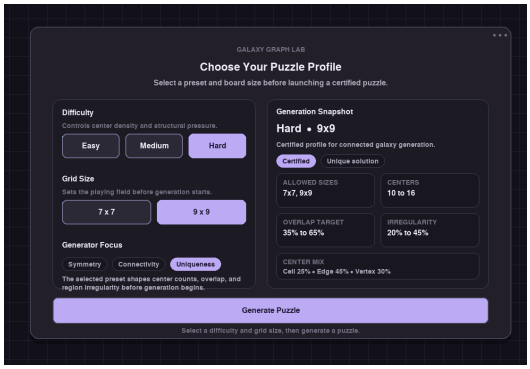

Fig. 1. Difficulty-selection logic used to map calibrated instance features back to one of the discrete difficulty labels.

At the experimental level, the implementation now separates two different dataset-oriented workflows. The first is `find_hard_threshold_limit()`, which answers the practical question of up to which square board size the instance remains solvable under a prescribed runtime threshold. This routine tests the `hard` family alone, starting from 7×7, increasing the side length by one unit at a time, generating one certified instance per size, and solving it with the exact backend `exact_flow`. The search terminates when the observed solve time first exceeds the chosen threshold, so that the largest size still solved within the threshold becomes an empirical upper bound for that execution regime.

The second workflow is the actual dataset-construction routine `generate_dataset()`. In its current default configuration, this function no longer performs the threshold search internally. Instead, it builds a fixed corpus over the square sizes 7×7, 8×8, 9×9, 10×10, and 11×11. For each of these sizes and for each of the three difficulty levels, the generator attempts to obtain a prescribed number of valid instances, twenty by default. Every accepted instance is generated under a size-aware difficulty profile, certified as a valid puzzle, and screened with the selected backend before being stored. Consequently, the dataset command has become a reproducible fixed-range corpus builder rather than a threshold-discovery procedure.

Because these board sizes extend the canonical profile sizes, the center-count interval must also be adapted to the board area. For a board with area $A = N \cdot M$, the dataset layer first computes the nominal center count $k_d^\star = \mathrm{round}(A/a_d)$, where a_d is the target area per center for difficulty d. Then, a tolerance parameter δ_d defines a narrow band around this nominal value. The lower bound is set to $k_d^{\min} = \max(\underline{k}_d, k_d^\star - \delta_d)$, so that it never falls below the canonical minimum $\underline{k}_d$. The upper bound starts from $k_d^\star + \delta_d$, but is clipped by the feasibility cap $\lfloor A/r_d \rfloor$, where r_d represents the minimum admissible area per region for that difficulty. Finally, the implementation enforces $k_d^{\max} \geq k_d^{\min}$. In this way, larger boards receive proportionally more centers, while the qualitative density and structural character of each difficulty family remain stable across sizes.

The manifest produced by the dataset layer records the active backend, the effective board sizes used in the corpus, the runtime filtering parameters, and the list of stored instances. On the solving side, `solve_instance()` solves a stored puzzle through a selected backend and exports compact diagnostic information. Similarly, `solve_dataset()` iterates this process over an entire dataset while aggregating timing, status, and backend-specific metadata. Because the solving command can execute `exact_flow`, `parallel_callback`, `heuristic_orbit`, or comparative multi-backend modes, the dataset layer acts not only as a persistence mechanism, but also as the empirical interface through which the three solution strategies can be compared under reproducible conditions.

I. Heuristic Model

A heuristic solution model is proposed for the system, implemented as a multi-start software procedure based on symmetric orbits. This approach follows a logic similar to the one previously described for instance generation. The central idea is that a valid galaxy must respect the $180°$ rotation around its center. Therefore, the minimal decision unit is not an individual cell u, but rather the orbit associated with cell u and center g, defined as

$$O_g(u) = \{u, \tau_g(u)\}. \tag{30}$$

In Eq. 30, $O_g(u)$ denotes the symmetric orbit induced by center g, and $\tau_g(u)$ is the cell obtained by applying the $180°$ rotation operator to u. Thus, assigning or removing a cell from a candidate galaxy is performed at the orbit level, ensuring that rotational symmetry is preserved throughout the heuristic construction process.

The solver starts from `PuzzleData`, which already contains the complete discrete geometry of the problem: cells, centers, admissible domains, kernels, symmetry twins, and the orthogonal adjacency graph. From this information, `HeuristicOrbitModel.from_puzzle_data()` constructs the data structures required by the heuristic search. The structure `orbit_by_center_and_cell` stores, for each center and each admissible cell, the corresponding symmetric orbit. The structure `unique_orbits_by_center` deduplicates these orbits so that the same symmetric pair is not evaluated more than once. The structure `candidate_centers_by_cell` records, for each cell, which centers may claim it according to admissibility. Finally, `target_area_by_center` estimates a target size for each galaxy by summing fractional contributions of the form $1/\,\mathrm{coverage}(u)$, so that a cell contributes less when it is admissible for many centers. The construction of heuristic solutions begins from the kernel of each center through `_initial_state()`. The internal state keeps two synchronized views of the partial assignment: `owner_by_cell`, which maps each cell to its current center, and `cells_by_center`, which maps each center to its current set of assigned cells. This allows the heuristic solver

to reason both at the cell level and at the galaxy, or region, level. It is important to emphasize that every kernel cell must be fixed to its corresponding galaxy from the beginning. Then, in `_construct_assignment()`, while unassigned cells remain, the solver generates the list of candidate orbits, scores them, randomly selects one among the best-ranked alternatives, and adds it to a region.

A candidate orbit is accepted only if it satisfies the predicate `_can_claim_orbit()`. This predicate verifies that all cells in the orbit are admissible for the selected center, that none of them is already occupied by another galaxy, that the orbit contributes at least one new cell, that it locally touches the current region so that growth occurs through the boundary, and that adding it preserves connectivity in the graph. In this way, the heuristic attempts to preserve the main constraints of the model throughout the incremental growth of the regions. The scoring logic implemented in `_candidate_score()` combines several criteria. The term `ambiguity_penalty` penalizes orbits that could be claimed by many centers, since higher ambiguity makes an assignment less reliable. The term `distance_penalty` penalizes orbits that are far from the geometric center of the galaxy. The term `boundary_growth_penalty` measures how much the orbit increases the boundary of the region. The term `size_penalty` penalizes deviations from the estimated target area. Conversely, `adjacency_bonus` rewards orbits that are well attached to the current boundary. The term `preferred_bonus` rewards orbits aligned with a preferred guide, whereas `preferred_conflict_penalty` penalizes orbits that contradict a mandatory preference. Finally, `avoid_penalty` penalizes assignments that coincide with a guide that should be avoided.

Therefore, the heuristic procedure does not merely fill the board. Instead, it biases the construction toward compact, symmetric, and connected regions, while preserving admissibility and respecting the orbit-based structure induced by the rotational geometry of the puzzle.

Within the set of candidate orbits, the solver randomly selects one among the five best-ranked candidates. The purpose of this controlled randomness is to prevent the heuristic solutions from becoming overly rigid and to allow the exploration of different constructions across multiple starts.

Once all board cells have been covered, a local improvement stage is applied through `_local_improvement()`. This procedure first identifies cells that are poorly aligned with `preferred_assignment_by_cell`. If no such cells are found, it then examines cells that still coincide with `avoid_assignment_by_cell`. For each candidate cell, the procedure attempts to move the corresponding orbit to another feasible center, revalidates the complete structure using `validate_assignment()`, and accepts the move only if it improves the guide-based objective.

In this way, the heuristic allows specific reassignment operations between regions, but only when the resulting assignment remains structurally valid. A final validation layer is then applied to verify that the solution produced by the heuristic model satisfies all required constraints.

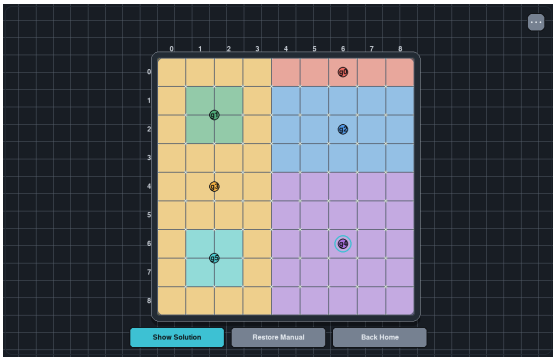

Fig. 2. Solved puzzle rendered in the interactive game interface.

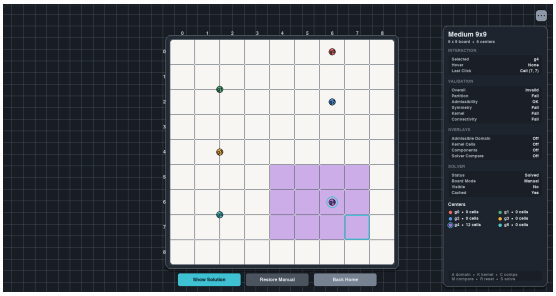

Fig. 3. Live structural-validation feedback in the interactive game interface.

Once a valid assignment has been produced by any backend, the same normalized solution object can be rendered by the interactive Pygame layer, allowing the user to visualize both the puzzle instance and the computed region ownership in the game interface, as shown in Fig. 2.

The same interactive layer also exposes the validation state of the current board configuration while the user edits a candidate solution, as shown in Fig. 3.

IV. Results

The experimental dataset was generated with the fixed-range pipeline described in the implementation section, rather than through the threshold-search utility. More precisely, the command-line generator produced square instances for all three difficulty levels on the range from 7×7 to 11×11, with 20 valid instances retained for each board size and each difficulty label. For every candidate, the pipeline executed profile-aware center placement, constructive region growth, partition closure, optional shape perturbation, and final certification with the exact flow backend. Only instances whose certified assignment passed full structural validation were stored. Since these board sizes extend the canonical profile sizes, the center count bounds were rescaled as a function of board area while preserving the qualitative targets attached to each difficulty profile, namely the cell/edge/vertex center mix, the admissible-domain overlap range, the irregularity target, and the minimum number of non-rectangular regions. The dataset was generated on square boards from 7×7 to 11×11, with 20 retained instances per size and difficulty, 64 generation retries per

TABLE I
DATASET-GENERATION PARAMETERS USED IN THE EXPERIMENTAL CAMPAIGN.

Parameter	E	M	H
Canonical center bounds	$[2, 5]$	$[6, 10]$	$[10, 16]$
Center mix (c/e/v)	$0.70/0.25/0.05$	$0.45/0.40/0.15$	$0.25/0.45/0.30$
Overlap target	$[0.00, 0.25]$	$[0.20, 0.45]$	$[0.35, 0.65]$
Irregularity target	$[0.00, 0.10]$	$[0.08, 0.22]$	$[0.20, 0.45]$
Min. non-rect. regions	0	1	2
Uniqueness flag	true	true	true
Target area a_d	16	11	8
Variation δ_d	1	1	2
Feasibility cap r_d	4	3	2

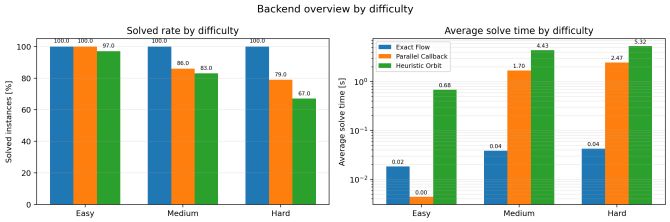

Fig. 4. Comparison of the three solver backends aggregated by difficulty.

seeded attempt, 24 outer seed sweeps, 16 seed blocks, and exact-flow screening/certification under the global public time limit of 0.5 seconds per solver call.

The size-aware scaling rule used to derive the center-count interval from the board area is described in the dataset-construction subsection of the methodology. Table I summarizes the numerical parameters that were used in the experimental campaign.

In the context of the obtained results, it is important to emphasize that, due to the way in which the instance generator is defined, all instances included in the test database necessarily have an exact solution that can be reached by the flow-based MILP solver within the maximum prescribed time limit of 0.5 seconds. This explains why all instances used in the experiments were solved exactly by this model.

However, the main interest lies in evaluating, over this same set of instances with certified exact solutions, the performance of two alternative solving approaches: the solver that uses a callback mechanism to enforce the connectivity constraint, and the heuristic solver. For this purpose, each method is assigned a maximum search time before declaring the instance unsolved: 10 seconds for the callback-based solver and 100 seconds for the heuristic model.

The performance of the generator is first evaluated in terms of difficulty selection. Fig. 4 shows that the observed behavior is consistent with the expected theoretical trend. For the three implemented solvers, the average time required to solve the instances increases as the difficulty level increases.

Since the same number of exactly solvable instances is used across all board dimensions, this result suggests that both the number of centers and the restrictions imposed on the base geometry of the system fulfill their design objective. In particular, these factors can be associated with an increase in

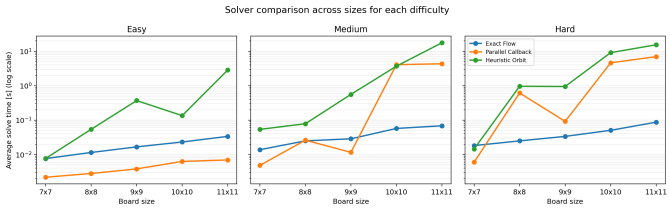

Fig. 5. Average solution time as a function of board size for each solver.

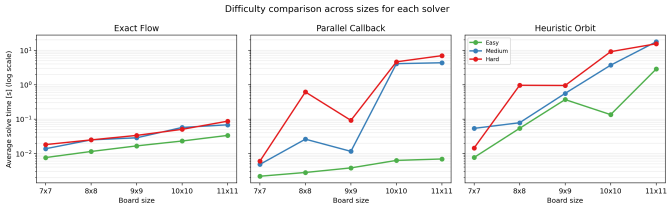

Fig. 6. Average solution time as a function of board size for each difficulty.

the difficulty of finding a valid solution for a given instance.

This interpretation is further supported by the fact that the number of instances solved within the assigned time limits decreases as the difficulty level increases. Such behavior is consistent with the expected dynamics of the difficulty-calibration process.

Fig. 4 also reveals an analytically relevant dynamic regarding the relationship between convergence speed and difficulty, particularly when comparing the callback-based solver with the exact flow-based solver. The callback-based solver exhibits a lower average solution time for easier instances. However, as instance difficulty increases, the exact solver is not only less affected by the increase in difficulty, but also achieves considerably faster convergence than the other two solvers.

This behavior can be explained by the methodological difference between the callback-based solver and the exact flow-based solver. As described in the methodology section, both approaches start from the same base constraint system, which captures all restrictive interactions of the puzzle except connectivity. In easy instances, two factors favor the callback-based approach. First, the cost of adding the flow variables required by the exact mathematical formulation of connectivity is significant when compared with the cost of solving the base system. Second, this additional cost is also significant when compared with the cost of iterating over base solutions until a connected solution is found.

This interpretation is reinforced by the results shown in Fig. 5, where larger and more complex instances tend to favor the exact flow-based formulation, whereas smaller and less complex instances tend to favor the callback-based solver. In these simpler cases, the structural complexity of the instance is not sufficient to compensate for the additional computational cost introduced by solving the enlarged matrix equality system associated with the exact connectivity formulation.

It is also relevant to analyze the behavior of the solving systems with respect to the board dimension for each instance.

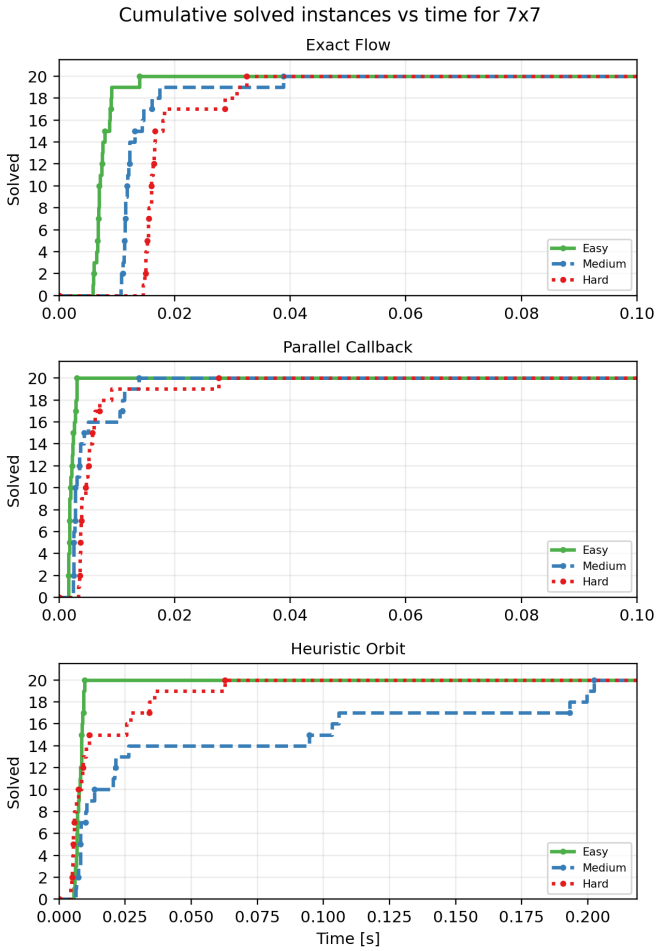

Fig. 7. Cumulative number of solved instances as a function of time for 7×7 boards.

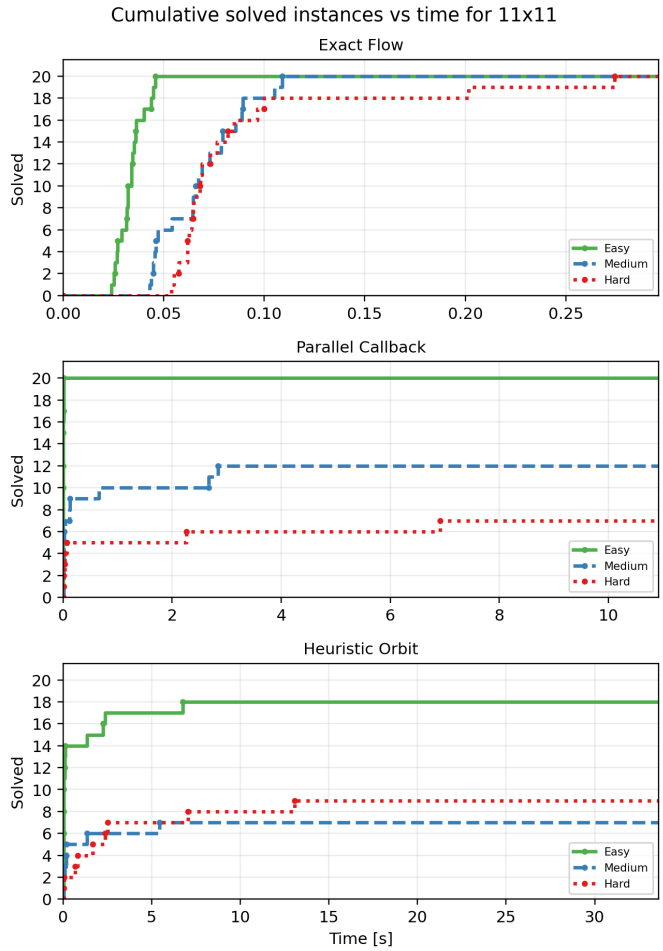

Fig. 8. Cumulative number of solved instances as a function of time for 11×11 boards.

In this case, the expected trend is observed: the average solution time increases as the board dimension becomes larger.

It is important to note that the technical separation associated with the easy difficulty level is clearer than the separation between medium and hard instances for each solver. Indeed, for some specific board dimensions, an overlap between the curves in Fig. 6 can be observed. Nevertheless, the relationship between the mean convergence times preserves the expected ordering across all solvers, with higher-difficulty instances generally requiring longer solution times.

Figs. 7–8 provide clear examples of the number of instances solved at each time instant for all solvers. The dynamics shown in these plots are consistent with the conclusions established above: the exact flow-based solver responds faster for instances with more complex structures, whereas it may be slower than the callback-based solver for simpler configurations.

In addition, these figures also show a reduction in the number of solutions found for more complex configurations when using solvers that involve heuristic components and are therefore not exact. They also reveal an increase in the average time required to reach a stable solved-state plateau as the difficulty level increases in most cases.

However, some particular dynamics may appear depending on the solver, usually due to geometric effects or overlaps between parameterization intervals in the generalized difficulty model. These effects can cause the observed curves to partially overlap, even when the overall trend remains consistent with the expected relationship between instance complexity, convergence time, and solver performance.

The last aspect to be discussed in the implementation results concerns the maximum board size for which reachable solutions persist under the exact MILP formulation within the established time threshold. Since the objective is to identify a practical solvability threshold, the analysis focuses on determining the board size at which hard instances reach this limit.

As shown in Fig. 9, the largest board size that remains consistently solvable for hard instances under the exact MILP configuration is 15×15. Nevertheless, for 16×16 boards, some generated instances can still be solved within the prescribed time threshold. However, in practice, no hard 17×17 instance was found to be solvable within the time limit. Therefore,

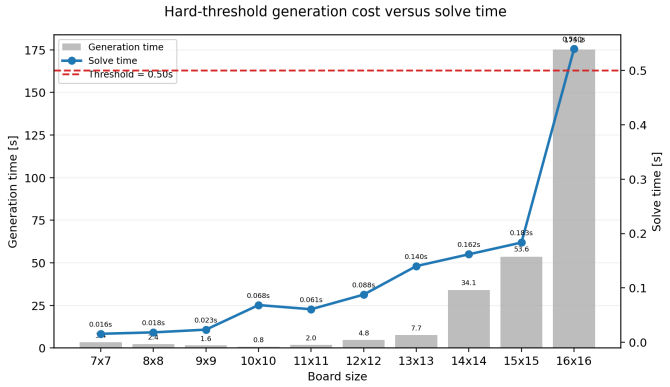

Fig. 9. Generation time and exact-flow solve time during the empirical `hard` threshold search.

within the context of the puzzle game, 17×17 hard instances are considered to exceed the valid solvability range established for the exact MILP solver.

V. Conclusion

The implementation of *Galaxy* demonstrates the feasibility of constructing a system for partitioning a discrete space from different methodological perspectives: from a mathematical formulation with relaxed constraints and callbacks, from an exact formulation based on flow constraints, or from a heuristic algorithm. In all cases, admissibility, kernel containment, rotational symmetry, and connectivity are encoded as the defining structural rules of a valid solution.

This framework is relevant not only for the development of the puzzle game itself, but also for other fields of knowledge, particularly in image segmentation tasks such as the separation of touching cell nuclei in histopathological or microscopy images.

At the implementation level, the project shows that the same compiled instance representation can support several complementary solving strategies. The callback-based backend demonstrates that connectivity can be enforced by iteratively rejecting disconnected base solutions. In addition, the orbit-based heuristic confirms that the geometric structure of the puzzle can be exploited to construct valid solutions without relying exclusively on exact optimization.

The results also show that the three implemented solving strategies correspond to different computational profiles. The callback-based method can be competitive on simpler instances because it avoids the fixed overhead introduced by the flow formulation. In contrast, the exact flow-based model becomes more stable as the structural difficulty of the instances increases. The heuristic solver, although not complete in the same sense as the exact formulation, still provides a practical alternative for interactive solving and comparative evaluation.

Overall, the project establishes a reusable framework for studying symmetry-constrained partition puzzles through graph theory, mixed-integer optimization, heuristic construction, and empirical benchmarking. Future work may extend this framework by improving the quality of the heuristic search, studying stronger connectivity cuts for the callback-based model, refining the difficulty-calibration pipeline, and expanding the experimental dataset toward larger instances and richer statistical analyses.

References

[1] G. Fertin, S. Jamshidi, and C. Komusiewicz, "Towards an algorithmic guide to Spiral Galaxies," *Theoretical Computer Science*, vol. 586, pp. 26–39, 2015, doi: 10.1016/j.tcs.2015.01.051.

[2] J. Shi and J. Malik, "Normalized Cuts and Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000, doi: 10.1109/34.868688.

[3] H. Validi, A. Buchanan, and E. Lykhovyd, "Imposing Contiguity Constraints in Political Districting Models," *Operations Research*, vol. 70, no. 2, pp. 867–892, 2022, doi: 10.1287/opre.2021.2141.

[4] F. K. Miyazawa, P. F. S. Moura, M. J. Ota, and Y. Wakabayashi, "Partitioning a graph into balanced connected classes: Formulations, separation and experiments," *European Journal of Operational Research*, vol. 293, no. 3, pp. 826–836, 2021, doi: 10.1016/j.ejor.2020.12.059.

[5] Y. Sun and B. Bhanu, "Reflection symmetry-integrated image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 9, pp. 1827–1841, 2012, doi: 10.1109/TPAMI.2011.259.

[6] L. Gorelick, O. Veksler, Y. Boykov, and C. Nieuwenhuis, "Convexity Shape Prior for Binary Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 2, pp. 258–271, 2017, doi: 10.1109/TPAMI.2016.2547399.

[7] N. Dey, S. M. Abulnaga, B. Billot, E. A. Turk, P. E. Grant, A. V. Dalca, and P. Golland, "AnyStar: Domain Randomized Universal Star-Convex 3D Instance Segmentation," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024, doi: 10.1109/WACV57701.2024.00742.